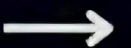
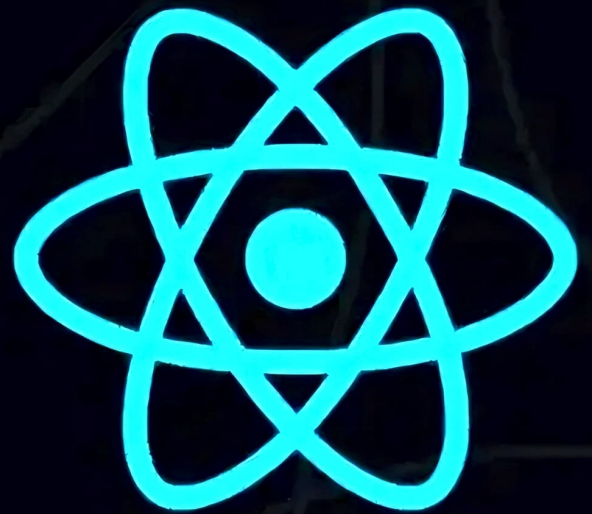


React Form





Creating a **form** in React involves different concepts such as **managing state**.

Let's create a simple form with a single input field and a submit button. We'll include **basic validation** to ensure the input is not empty.

Lets see the code, followed by an explanation of each step and the use case of state.

Simple Form Example

```
import React, { useState } from 'react';

const SimpleForm = () => {
  // State to manage the input value and the error message
  const [inputValue, setInputValue] = useState('');
  const [error, setError] = useState('');

  // Handle input change
  const handleChange = (e) => {
    setInputValue(e.target.value);
  };

  // Handle form submission
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent the default form
    submission
    if (inputValue.trim() === '') {
      setError('Input cannot be empty');
    } else {
      setError('');
      console.log('Form submitted with input:', inputValue);
      // Here you can handle the form submission, e.g., send
      data to a server
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>Input</label>
        <input
          type="text"
          value={inputValue}
          onChange={handleChange}
        />
        {error && <p>{error}</p>}
      </div>
      <button type="submit">Submit</button>
    </form>
  );
};

export default SimpleForm;
```



Steps and Use Case of State

1. Import React and useState

```
import React, { useState } from 'react';
```

We import `React` and the `useState` hook to manage the form's state.

2. Initialize State

```
const [inputValue, setInputValue] = useState('');  
const [error, setError] = useState('');
```

`inputValue` holds the current value of the input field. `error` holds the `error message` if the input is invalid.

3. Handle Input Change

```
const handleChange = (e) => {  
  setInputValue(e.target.value);  
};
```

The `handleChange` function updates `inputValue` with the new value from the input field.

It uses the `setInputValue` function to update the state whenever the user types into the input field.

4. Handle Form Submission

```
const handleSubmit = (e) => {  
  e.preventDefault(); // Prevent the default form submission  
  if (inputValue.trim() === '') {  
    setError('Input cannot be empty');  
  } else {  
    setError('');  
    console.log('Form submitted with input:', inputValue);  
    // Here you can handle the form submission, e.g., send  
    data to a server  
  }  
};
```

The `handleSubmit` function prevents the default form submission behavior.

It checks if the `inputValue` is empty. If it is, it sets an error message using `setError`.

If the input is not empty, it clears the error message and `logs the input value`. You can replace the `console.log` statement with code to handle the form submission, such as sending data to a server.

5. Render the Form

```
return (  
  <form onSubmit={handleSubmit}>  
    <div>  
      <label>Input</label>  
      <input  
        type="text"  
        value={inputValue}  
        onChange={handleChange}  
      />  
      {error && <p>{error}</p>}  
    </div>  
    <button type="submit">Submit</button>  
  </form>  
);
```

The form is rendered with an **input field** and a **submit button**.

The value of the input field is bound to **inputValue**, and the **onChange** event is handled by **handleChange**.

If there is an error, it is displayed below the input field.

The form's **onSubmit** event is handled by **handleSubmit**.

